Cache-Craft: Autonomous Semantic Cache Router

A Multi-Tier Semantic Caching Middleware for LLM Inference

Aakash Bharatbhai Jayani (23SE02IT140) | Mentor: Mr. Anurag Yadav

B.Tech Information Technology, 7th Semester | Department of Information Technology

P P Savani University, School of Engineering

School of Engineering

Abstract

Cache-Craft answers repeated or **paraphrased** LLM prompts from a three-tier cache (process memory, Redis, PostgreSQL + pgvector) instead of re-calling the model. On a 1,200-query benchmark it reached **100% precision**, **96.2% recall** and **3.12× faster** mean latency (**68% lower**) than the stated uncached baseline.

Introduction

- **Cost and latency:** LLM calls take about 2–4 s and are billed per token.
- **Exact-match fails:** "What is a semantic cache?" and "Explain semantic caching" hash differently, so paraphrases always miss.
- **Fragile providers:** cloud APIs return 429/503 under load.

Research question: can semantic matching reuse responses at high precision, in real time?

Gap: GPTCache, RedisVL and Towhee are single-tier vector stores with no exact-hash bypass, multi-turn isolation or built-in dashboard (per the report's review).

Objectives

- Build an L0 → L1 → L2 multi-tier cache path.
- Tune a cosine threshold τ for 100% precision.
- Add Gemini → Ollama fallback and multi-turn context isolation.
- Ship a Next.js dashboard and a benchmark suite.

Materials

Backend	Python, FastAPI
Frontend	Next.js 16, React 19, TypeScript, Tailwind
Cache tiers	Memory (L0), Redis (L1), PostgreSQL 16 + pgvector HNSW (L2)
Embeddings	all-MiniLM-L6-v2, 384-d, cosine
Inference	Gemini; Ollama llama3.2 (local)
Evaluation	NumPy benchmark, pytest
Deploy	Docker Compose; Vercel + Railway

Methodology

1 · Normalise and hash the prompt
SHA-256 key; follow-ups bound to context

▼ miss

2 · L0 memory → L1 Redis
Exact hit returns in about 1–2 ms

▼ miss

3 · L2 pgvector semantic search
384-d HNSW; hit if similarity ≥ τ

▼ no entry ≥ τ

4 · Inference on miss
Gemini → Ollama fallback

▼

5 · Store and stream telemetry
Dual TTL; REST + SSE metrics

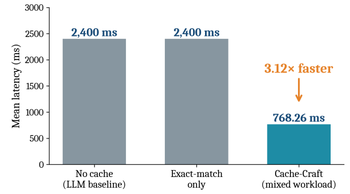
Request routing through exact-match tiers, semantic search and configured inference.

Evaluation design

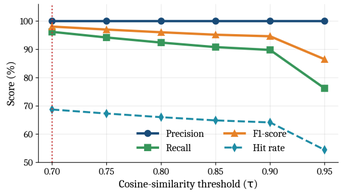
- **Corpus (1,200):** 500 seeds, 500 paraphrases, 200 unrelated controls.
- **Protocol:** seed with 500, score 700 cases at τ = 0.70–0.95.
- **Metrics:** TP/FP/FN/TN, precision, recall, F1, hit rate, latency, estimated cost.

Results & Discussion

100%	96.2%	0.9806	68.71%	3.12×
precision	recall	F1-score	hit rate	faster



Mean latency over 700 requests (baseline is a stated figure). Reported hits: L0 1.84 ms, L2 25.33 ms.

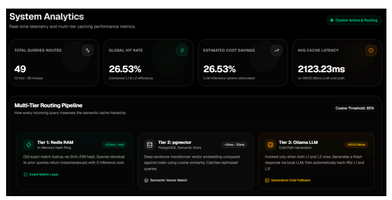


Precision stays 100% for every τ; recall drops to 76.2% at τ = 0.95.

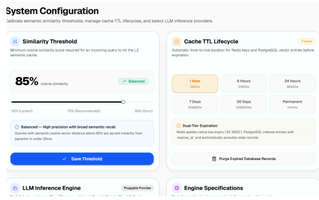
System	Hit rate	Mean latency	Speed-up
Direct LLM (no cache)	0%	2,400 ms	1.0×
Exact-match only	0%	2,400 ms	1.0×
Cache-Craft (τ = 0.70)	**68.71%**	**768.26 ms**	**3.12×**

Exact-match caching scores 0% on paraphrased queries; semantic routing serves 68.71% of requests from cache.

Discussion. At τ = 0.70 the confusion counts are TP 481, FP 0, FN 19, TN 200: the best F1 of the thresholds tested, with no false hit on any unrelated control. The 68.71% **API-cost saving** is a model estimate (each hit avoids one upstream call), separate from the measured latency gain.



Live analytics dashboard with the L0/L1/L2 routing pipeline



Settings: threshold slider, TTL, inference engine

Conclusion

Combining exact-match tiers with vector search cuts repeated LLM calls while holding precision on this corpus, beating the targets of ≥ 60% hit rate, ≥ 95% precision and ≥ 2.5× speed-up. **Applications:** chatbots, RAG services, support and documentation assistants.

Project Phases

Phase 1 · Wk 1–4: research and design

Phase 2 · Wk 5–8: baseline prototype, τ proof

Phase 3 · Wk 9–12: pgvector HNSW, dual LLM, UI v1

Phase 4 · Wk 13–16: multi-tier router, benchmark, deploy

Challenges & Fixes

- **Follow-up collision:** follow-ups scored ≈ 0.97 vs. the parent query; partition filtering now isolates them.
- **Gemini 503/429:** best-effort Ollama fallback.
- **Slow benchmark:** NumPy matrix scoring cut it from ~30 s to ~2 s.
- **Stale answers:** dual TTL in Redis and PostgreSQL.

Recommendations

Limitations

- Curated English corpus; cost savings are estimated, not billed.
- **Remote GPU authorization is not implemented.**
- 12 automated tests; the 163-scenario matrix is a manual record.

Future work

- Adaptive thresholds, multilingual embeddings.
- OAuth2 / RBAC and tenant isolation.
- Redis cluster, multi-modal caching.

Acknowledgements

Thanks to Mr. Anurag Yadav, the Department of IT and P P Savani University, School of Engineering, for guidance and support.

Refs: Reimers & Gurevych (2019), SBERT; Malkov & Yashunin (2018), HNSW; pgvector docs.

Code: github.com/aakashjayani19352-wq/Cache-Craft
Demo: cache-craft.vercel.app